

Offline Cache Instructions (Feature 5)

This document explains how to implement offline caching for the coin list so the app can render the last successful data when the API is unreachable.

Goal

When the app starts: - If the API is reachable, fetch fresh data and update the UI. - If the API fails (network error, rate limit, etc.), load the last cached coin list from `localStorage` and render it instead.

Files Involved

- `src/hooks/useFetchCoins.js`
- `src/services/coinService.js`
- `src/pages/DashboardPage.js` (only if you want to surface a UI banner)

Step-by-Step Implementation

1. Add local cache keys and helpers
 - Store the last successful coin list in `localStorage` under a key like `coinCache`.
 - Store a timestamp under a key like `coinCacheUpdatedAt`.
2. Update the hook to write cache on success
 - When `fetchCoinsData()` succeeds, `setCoins(data)` and save to `localStorage`.
3. Update the hook to read cache on failure
 - On `catch`, attempt to parse cached data from `localStorage`.
 - If it exists, `setCoins(cachedData)`.
 - Optionally track a `usedCache` boolean state for UI messaging.
4. Optional UI banner for transparency
 - If `usedCache` is true, show a small banner like “Offline mode: showing last updated data.”
 - Display the last updated timestamp if stored.
5. Optional: cache on interval updates
 - Each successful poll should refresh the cache so it’s always recent.

Example Pseudocode

```
// in useFetchCoins.js
const CACHE_KEY = 'coinCache';
const CACHE_TS_KEY = 'coinCacheUpdatedAt';

try {
  const data = await fetchCoinsData();
  setCoins(data);
```

```

localStorage.setItem(CACHE_KEY, JSON.stringify(data));
localStorage.setItem(CACHE_TS_KEY, new Date().toISOString());
setUsedCache(false);
} catch (err) {
  const cached = localStorage.getItem(CACHE_KEY);
  if (cached) {
    setCoins(JSON.parse(cached));
    setUsedCache(true);
  }
}
}

```

Image Reference

This image is included as a visual placeholder for documentation consistency:

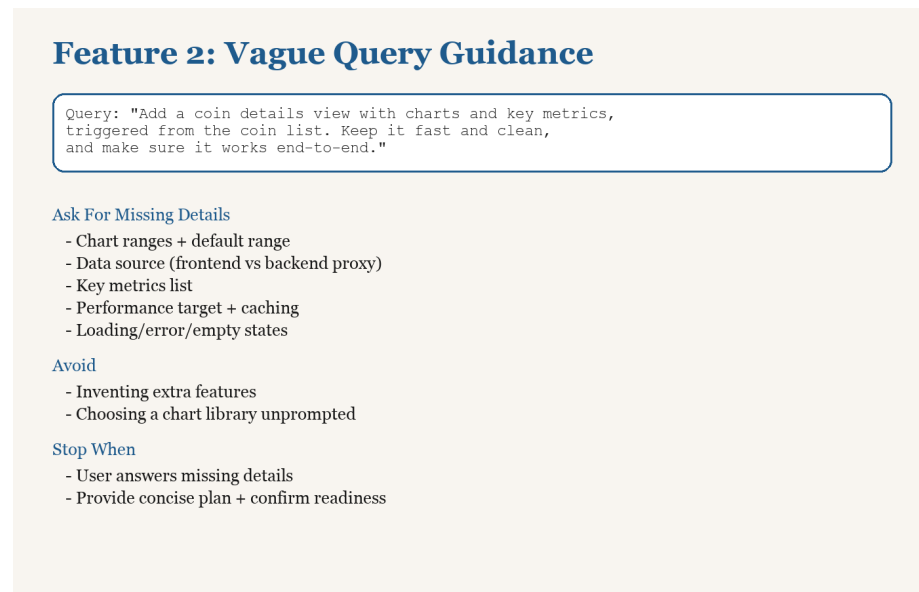


Figure 1: Offline Cache Feature Diagram